

FastAPI: Basics to Production

A complete, hands-on guide to designing, building, securing, testing and deploying a production-grade FastAPI service

Part 1 — Introduction & Setup

1.1 What is FastAPI and why use it

FastAPI is a modern, high-performance Python web framework for building APIs, built on top of standard Python type hints. It uses Starlette for the web/ASGI layer and Pydantic for data validation, which gives you automatic request validation, serialization, and interactive documentation for free.

- Performance: one of the fastest Python frameworks available, comparable to Node.js and Go, because it is built on ASGI (Asynchronous Server Gateway Interface) and supports native `async/await`.
- Type-hint driven: function parameters and Pydantic models double as validation rules and documentation — no separate schema files needed.
- Automatic docs: interactive Swagger UI (`/docs`) and ReDoc (`/redoc`) are generated automatically from your code.
- Dependency Injection: a first-class, simple system (Depends) for shared logic such as DB sessions, auth, and pagination.
- Standards-based: fully compatible with OpenAPI and JSON Schema, which makes client SDK generation and contract testing easy.

1.2 Installation

FastAPI needs an ASGI server to actually run — Uvicorn is the standard choice for development, and Uvicorn workers managed by Gunicorn is the standard for production (covered in Part 5).

```
# Recommended: use a virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# Core install
pip install "fastapi[standard]"  # includes uvicorn, pydantic, python-multipart, etc.

# Or minimal install + explicit server
pip install fastapi uvicorn[standard]
```

1.3 Your first FastAPI app

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="My API", version="1.0.0")

@app.get("/")
def read_root():
    return {"message": "Hello, Somnath!"}

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Run it with the development server (auto-reload on code changes):

```
uvicorn main:app --reload
# Visit http://127.0.0.1:8000/docs for interactive Swagger UI
# Visit http://127.0.0.1:8000/redoc for ReDoc documentation
```

Tip: Every code sample in this guide is self-contained. Copy any section into a file and run it with uvicorn to see it work before moving to the next concept.

Part 2 — Core Basics

2.1 Path & query parameters

FastAPI infers parameter source (path vs query) automatically from the function signature. Type hints are enforced and invalid input returns an automatic HTTP 422 with a clear error body.

```
from fastapi import FastAPI
from typing import Optional

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int, q: Optional[str] = None, active: bool = True):
    # item_id -> path parameter (required, must be int)
    # q       -> optional query parameter, e.g. /items/5?q=hello
    # active  -> query parameter with a default value
    result = {"item_id": item_id, "active": active}
    if q:
        result["q"] = q
    return result
```

2.2 Request body with Pydantic models

Define the shape of incoming JSON using a Pydantic model. FastAPI validates the body, converts types, and shows the schema in /docs automatically.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float = Field(gt=0, description="Price must be greater than 0")
    tax: float | None = None

@app.post("/items/")
def create_item(item: Item):
    total = item.price + (item.tax or 0)
    return {"item": item, "price_with_tax": total}
```

2.3 Response models & status codes

Use `response_model` to control exactly what is returned (e.g., hide a password field) and `status_code` to set the correct HTTP status for the operation.

Use two different Pydantic models — one for what comes IN, one for what goes OUT — so sensitive or internal fields never accidentally leak to the client.

```
from fastapi import FastAPI, status
from pydantic import BaseModel

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str          # comes in, but should never go out

class UserOut(BaseModel):
    username: str          # only this is returned to the client

@app.post("/users/", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def create_user(user: UserIn):
    # ... save user, hash password, etc.
    return user # FastAPI filters the response down to UserOut fields
```

2.4 Error handling with HTTPException

This is **FastAPI's** standard, built-in way to return proper **HTTP error responses** instead of crashing or returning misleading success codes.

HTTPException is a special exception class FastAPI understands natively — when you raise it anywhere inside a route function, FastAPI catches it automatically and converts it into a proper JSON error response. You don't need any try/except wrapper yourself.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
fake_db = {"1": "Widget"}

@app.get("/items/{item_id}")
def get_item(item_id: str):
    if item_id not in fake_db:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item_id": item_id, "name": fake_db[item_id]}
```

2.5 Quick reference — common parameter types

Location	How to declare	Example
Path	Function parameter matching {name} in route	@app.get("/items/{item_id}") def f(item_id: int)
Query	Function parameter NOT in the path	def f(skip: int = 0, limit: int = 10)
Body	Pydantic BaseModel parameter	def f(item: Item)
Header	Header(...) default	def f(user_agent: str = Header(None))

Location	How to declare	Example
	Header() tells FastAPI "look in the request's HTTP headers section for a value. Python parameter name (snake_case) → auto-converted to Hyphen-Case → matched against the real header key.	
Cookie	Cookie(...) default Cookie() tells FastAPI "parse the browser's Cookie: request header, and extract the value for the cookie named exactly this." Here, unlike Header(), there's no case/character	def f(session_id: str = Cookie(None))

Best Practice: Keep one Pydantic model per purpose — e.g., ItemCreate, ItemUpdate, ItemResponse — instead of reusing a single model everywhere. This avoids accidentally exposing internal fields and makes validation rules precise per operation.

Part 3 — Intermediate Concepts

3.1 Dependency Injection (Depends)

Dependency Injection (DI) is FastAPI's system for saying: "before running my endpoint function, run this other function first, and hand me its result as a parameter." Those reusable "other functions" are called **dependencies**, and you wire them in using **Depends()**

Dependencies are reusable functions FastAPI calls for you before the endpoint runs. They are the backbone of clean FastAPI apps — used for DB sessions, current-user resolution, pagination, and shared validation.

```
from fastapi import Depends, FastAPI

app = FastAPI()

def common_pagination(skip: int = 0, limit: int = 20):
    return {"skip": skip, "limit": limit}

@app.get("/items/")
def list_items(pagination: dict = Depends(common_pagination)):
    return {"pagination": pagination, "items": []}

# Dependencies can depend on other dependencies (chained), and FastAPI
# caches the result within a single request so it's only computed once.
```

Depends can also just *enforce* something, without returning a value you use.

```
def verify_admin(user: dict = Depends(get_current_user)):
```

```

    if not user.get("is_admin"):
        raise HTTPException(status_code=403, detail="Admins only")

@app.delete("/items/{item_id}", dependencies=[Depends(verify_admin)])
def delete_item(item_id: int):
    return {"deleted": item_id}

```

Notice `dependencies=[Depends(verify_admin)]` in the route decorator itself (not as a function parameter) — this runs the check but doesn't inject any value into `delete_item`. Use this when you only care about the side effect (raise if unauthorized) and don't need the return value.

3.2 Modular apps with APIRouter

As soon as you have more than a handful of endpoints, split them into routers by feature/domain. This is the single highest-leverage habit for keeping a FastAPI codebase maintainable.

- **APIRouter** lets you group related endpoints (e.g., users, items, auth) into separate files instead of one large `main.py`.
- Each router is created with `APIRouter(prefix="/items", tags=["items"])`, where `prefix` auto-adds the path and `tags` groups it in Swagger docs.
- Routes inside use `@router.get(...)` / `@router.post(...)` just like `@app.get(...)`, but scoped to that router.
- In `main.py`, you register each router with `app.include_router(router)` to activate its routes on the main app.
- It improves maintainability, avoids merge conflicts in teams, and keeps large FastAPI projects modular and scalable.

```

# app/routers/items.py
from fastapi import APIRouter

router = APIRouter(prefix="/items", tags=["items"])

@router.get("/")
def list_items():
    return []

@router.get("/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}

# app/main.py
from fastapi import FastAPI
from app.routers import items, users, auth

app = FastAPI()
app.include_router(auth.router)
app.include_router(users.router)
app.include_router(items.router)

```

Each router file independently defines its own router object with its own routes and prefix; `main.py` imports each file as a module and uses `.router` to reach into it and pull out that specific object, then

`app.include_router(...)` merges those routes into the live application — three separate, isolated pieces assembled into one working API.

3.3 Middleware & CORS

- **Middleware** is code that runs on every request/response passing through the app — used for logging, timing, adding custom headers, etc., via `@app.middleware("http")`.
- **CORS (Cross-Origin Resource Sharing)** is a browser security rule that blocks a webpage on one domain from calling an API on a different domain, unless the API explicitly allows it.
- FastAPI enables CORS using `CORSMiddleware`, configured with `allow_origins`, `allow_methods`, `allow_headers`, and `allow_credentials`.
- CORS is a **browser-enforced** guardrail only — it doesn't stop direct server-to-server calls (e.g., via `curl`/`Postman`), so it must always be paired with real authentication.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import time

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://myapp.example.com"], # never "*" with credentials
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
)

@app.middleware("http")
async def add_process_time_header(request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.perf_counter() - start)
    return response
```

Important: CORS is a browser-enforced guardrail, not server-side security. It stops malicious webpages from reading your API's response in a victim's browser — it does NOT stop direct calls via `curl`/`Postman`/server-to-server. Always pair it with real authentication (Part 4).

- Middleware handles every HTTP request and response.
- Register using `@app.middleware("http")`.
- `request` contains incoming request data.
- `call_next` continues request processing.
- Code before `call_next` handles the request.
- Code after `call_next` handles the response.
- `return response` sends the final response.

- Middleware uses stack-based execution.
- Last registered runs first for requests.
- After the endpoint returns, the response travels through middleware in reverse order.

```
app.middleware("http")(middleware_a)
app.middleware("http")(middleware_b)
```

- Request sequence: B → A → Endpoint
- Response sequence: Endpoint → A → B
- Register middleware centrally in `main.py`.

3.4 Background tasks

- BackgroundTasks lets an endpoint return a response immediately while some lightweight work (e.g., sending an email, writing a log) runs after the response is sent.
- You add it as a parameter (`background_tasks: BackgroundTasks`) and queue work with `background_tasks.add_task(function, *args)`.
- It's only for light, fire-and-forget jobs — anything heavier or that must survive a server restart should use a real task queue (Celery, RQ, Azure Queue Storage) instead

```
from fastapi import BackgroundTasks, FastAPI

app = FastAPI()

def send_welcome_email(email: str):
    print(f"Sending welcome email to {email}...")

@app.post("/signup")
def signup(email: str, background_tasks: BackgroundTasks):
    background_tasks.add_task(send_welcome_email, email)
    return {"message": "Signup received"}  # returns immediately
```

3.5 File uploads

- FastAPI creates the API, while `UploadFile` and `File` handle uploaded files.
- `@app.post("/upload")` creates a POST API endpoint at `/upload` for uploading a file.
- `UploadFile = File(...)` accepts the uploaded file. `File(...)` means the file is mandatory.
- `await file.read()` reads the file, and the API returns its filename, size in bytes, and content type as JSON.

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
```

```

    contents = await file.read()
    return {"filename": file.filename, "size_bytes": len(contents), "content_type":
file.content_type}

```

3.6 Async vs. sync endpoints

Use `async def` when your endpoint awaits I/O (DB calls with an async driver, HTTP calls with `httpx.AsyncClient`, etc.). Use a regular `def` when your code is synchronous/blocking (e.g., a sync ORM call or CPU-bound work) — FastAPI automatically runs sync endpoints in a threadpool so they don't block the event loop.

Situation	Use
Async DB driver (asyncpg, motor)	<code>async def</code> + <code>await</code>
Sync DB driver (psycopg2, sync SQLAlchemy)	<code>def</code> (FastAPI threadpools it)
Calling another API with <code>httpx.AsyncClient</code>	<code>async def</code> + <code>await</code>
CPU-heavy work (image processing, ML inference)	<code>def</code> , or offload to a worker/queue

Part 4 — Advanced: Building a Real Backend

4.1 Configuration management with Pydantic Settings

Never hard-code secrets or environment-specific values. Pydantic's `BaseSettings` reads from environment variables (and a `.env` file locally), validates types at startup, and is cached with `lru_cache` so it's read once.

```

# app/core/config.py
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", case_sensitive=False)

    app_name: str = "My API"
    environment: str = "development"
    database_url: str
    secret_key: str
    access_token_expire_minutes: int = 30
    algorithm: str = "HS256"
    allowed_origins: list[str] = ["*"]

    @lru_cache
    def get_settings() -> Settings:
        return Settings()

# Usage in an endpoint:
# from app.core.config import get_settings
# def endpoint(settings: Settings = Depends(get_settings)): ...

```

4.2 Database integration (SQLAlchemy 2.0, async)

For production, use SQLAlchemy's async engine with a connection-pool-aware session dependency, plus Alembic for versioned migrations.

```
# app/db/database.py
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession
from app.core.config import get_settings

settings = get_settings()
engine = create_async_engine(settings.database_url, pool_size=20, max_overflow=10)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session

# app/models/user.py
from sqlalchemy.orm import Mapped, mapped_column
from app.db.database import Base

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(unique=True, index=True)
    hashed_password: Mapped[str]
    is_active: Mapped[bool] = mapped_column(default=True)

# app/routers/users.py
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.db.database import get_db
from app.models.user import User

router = APIRouter(prefix="/users", tags=["users"])

@router.get("/{user_id}")
async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    return result.scalar_one_or_none()
```

Migrations: Use Alembic (alembic init alembic) to version every schema change. Never rely on `Base.metadata.create_all()` in production — it can't alter existing tables safely.

4.3 Authentication: OAuth2 password flow + JWT

The standard FastAPI pattern: collect username/password via `OAuth2PasswordRequestForm`, verify against a hashed password, and issue a signed, short-lived JWT access token. Every protected route then depends on a function that decodes and validates that token.

```
# app/core/security.py
from datetime import datetime, timedelta, timezone
import jwt
from pwdlib import PasswordHash
from app.core.config import get_settings
```

```

settings = get_settings()
password_hash = PasswordHash.recommended() # Argon2 – modern, FastAPI-recommended

def verify_password(plain: str, hashed: str) -> bool:
    return password_hash.verify(plain, hashed)

def hash_password(plain: str) -> str:
    return password_hash.hash(plain)

def create_access_token(subject: str, expires_minutes: int | None = None) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=expires_minutes or settings.access_token_expire_minutes
    )
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)

# app/routers/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
import jwt
from app.core.security import verify_password, create_access_token
from app.core.config import get_settings

router = APIRouter(tags=["auth"])
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
settings = get_settings()

@router.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = await authenticate_user(form_data.username, form_data.password) # your DB lookup
    if not user:
        raise HTTPException(status_code=401, detail="Incorrect username or password",
                            headers={"WWW-Authenticate": "Bearer"})
    token = create_access_token(subject=user.email)
    return {"access_token": token, "token_type": "bearer"}

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
        email = payload.get("sub")
        if email is None:
            raise HTTPException(status_code=401, detail="Invalid token")
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
    return await get_user_by_email(email) # your DB lookup

# Protecting a route:
# @app.get("/me")
# def read_me(current_user = Depends(get_current_user)):
#     return current_user

```

- Never store plaintext passwords — always hash with Argon2 via `pwdlib` (the current FastAPI-recommended library; it avoids a well-known `passlib/bcrypt` version-compatibility bug).
- Keep access tokens short-lived (15–30 min) and use a separate, longer-lived refresh token if you need persistent sessions.
- Pin the JWT algorithm explicitly (e.g., HS256 or RS256) to block "alg=none" bypass attacks.

- If your organization uses Microsoft Entra ID (Azure AD), prefer validating Entra-issued tokens over rolling your own — see the Deployment part for the pattern you've already documented.

4.4 Custom exception handlers

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

class OutOfStockError(Exception):
    def __init__(self, item: str):
        self.item = item

@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(request: Request, exc: OutOfStockError):
    return JSONResponse(status_code=409, content={"detail": f"{exc.item} is out of stock"})
```

4.5 Testing with pytest

```
# tests/conftest.py
import pytest
from httpx import AsyncClient, ASGITransport
from app.main import app

@pytest.fixture
async def client():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        yield ac

# tests/test_users.py
import pytest
@pytest.mark.anyio
async def test_read_root(client):
    response = await client.get("/")
    assert response.status_code == 200
    assert response.json()["message"]
```

Best Practice: Set up your async test client from day one (httpx.AsyncClient + ASGITransport) — retrofitting tests after the app grows is far more painful.

4.6 Structured logging

```
import logging, sys

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(name)s %(message)s',
    stream=sys.stdout, # containers should log to stdout, not files
)
logger = logging.getLogger("app")

# In an endpoint:
# logger.info("user_login", extra={"user_id": user.id})
```

Part 5 — Production Deployment

5.1 Recommended project structure

This is the structure the sample project (delivered alongside this guide) follows — a feature-friendly layout that scales from a solo project to a multi-developer team.

```

myapi/
├── app/
│   ├── main.py           # FastAPI() app factory, includes routers
│   ├── core/
│   │   ├── config.py     # Pydantic Settings
│   │   └── security.py   # password hashing, JWT
│   ├── db/
│   │   └── database.py   # engine, session, Base
│   ├── models/           # SQLAlchemy ORM models
│   ├── schemas/          # Pydantic request/response models
│   └── routers/          # one file per feature (auth, users, items)
├── alembic/              # DB migrations
├── tests/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt / pyproject.toml
└── .env.example          # never commit the real .env

```

5.2 Dockerizing the app

```

# Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

# Gunicorn manages multiple Uvicorn worker processes for production
CMD ["gunicorn", "app.main:app", \
    "--workers", "4", \
    "--worker-class", "uvicorn.workers.UvicornWorker", \
    "--bind", "0.0.0.0:8000", \
    "--timeout", "120"]

```

Number of workers: a common rule of thumb is $(2 \times \text{CPU cores}) + 1$ for mixed workloads, or simply one worker per core for purely async, I/O-bound apps. Always load-test to confirm.

5.3 docker-compose for local production-like testing

```

# docker-compose.yml
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env

```

```

    depends_on: [db]
db:
  image: postgres:16
  environment:
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: ${DB_NAME}
  volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:

```

5.4 Environment variables & secrets

- Never commit `.env` — commit only `.env.example` with dummy placeholder values.
- In Azure, store secrets in Azure Key Vault and reference them as app settings, or inject via managed identity — never bake secrets into the Docker image.
- Use different `secret_key` / `database_url` per environment (dev, staging, prod).

5.5 CI/CD with GitHub Actions

```

# .github/workflows/deploy.yml
name: Build and Deploy
on:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/docker-login@v1
        with:
          login-server: ${ secrets.ACR_LOGIN_SERVER }
          username: ${ secrets.ACR_USERNAME }
          password: ${ secrets.ACR_PASSWORD }
      - run: |
          docker build -t ${ secrets.ACR_LOGIN_SERVER }/myapi:${ secrets.github.sha } .
          docker push ${ secrets.ACR_LOGIN_SERVER }/myapi:${ secrets.github.sha }
      - uses: azure/webapps-deploy@v3 # or Azure Container Apps deploy action
        with:
          app-name: my-fastapi-app
          images: ${ secrets.ACR_LOGIN_SERVER }/myapi:${ secrets.github.sha }

```

5.6 Choosing between Azure App Service and Container Apps

You already have a detailed cheat sheet on this ([FastAPI_Azure_Deployment_Cheatsheet.pdf](#), shared yesterday) covering App Service vs. Container Apps, VNet/networking, CORS placement,

and the 5-layer security stack (Entra ID JWT, App Roles, client wiring, network gate, hardening). The quick decision rule from that reference:

Question	Choose
Single deployable API, steady traffic, simplest learning curve?	App Service
Multiple microservices, need scale-to-zero, must ship Docker images?	Container Apps
Need GPU for an AI model?	Container Apps (dedicated)

Reference: See your own [FastAPI_Azure_Deployment_Cheatsheet.pdf](#) for the full 3-page breakdown of required vs. optional components, VNet/subnet sizing, and the complete 5-layer security stack (Entra ID JWT → App Roles → client wiring → network gate → hardening) — it directly complements this guide for the Azure-specific parts of deployment.

5.7 Health checks, monitoring & observability

```
@app.get("/health")
def health():
    return {"status": "healthy"}

@app.get("/health/db")
async def health_db(db: AsyncSession = Depends(get_db)):
    await db.execute(text("SELECT 1"))
    return {"status": "healthy", "db": "reachable"}
```

- Wire /health into your container platform's liveness/readiness probe (Container Apps health probes, App Service health check path).
- Send structured logs and traces to Azure Application Insights (or your APM of choice) — capture request latency, error rate, and dependency (DB/HTTP) call duration.
- Add correlation/request IDs to every log line so you can trace a single request across services.

5.8 Production security checklist

- Enforce HTTPS/TLS everywhere; never accept plaintext HTTP in production.
- Rate-limit per-user/IP (e.g., via slowapi or an API Management / App Gateway layer) to blunt abuse.
- Use Pydantic's extra="forbid" on request models to reject unexpected fields.
- Keep separate Create/Update/Response Pydantic schemas so internal fields never leak.
- Rotate secrets regularly; store them in Key Vault, not environment files checked into git.
- Pin dependency versions (requirements.txt / poetry.lock) and scan for vulnerabilities in CI.
- Add an audit log for sensitive operations (who did what, when).

Part 6 — Your Learning Roadmap

A suggested order to actually build the production project mentioned in your request, using the sample starter project delivered alongside this guide:

1. Week 1 — Basics: build 5-10 endpoints (CRUD) with Pydantic models, no database yet (in-memory dict). Get comfortable with /docs.

2. Week 2 — Database: wire up SQLAlchemy async + PostgreSQL, add Alembic migrations, convert your in-memory CRUD to real DB calls.
3. Week 3 — Auth: add OAuth2 + JWT login, protect routes with Depends(get_current_user), add role-based access if needed.
4. Week 4 — Hardening: add tests (pytest + httpx), structured logging, custom exception handlers, and a health check.
5. Week 5 — Containerize & deploy: Dockerfile + docker-compose locally, then push to Azure Container Registry and deploy via App Service or Container Apps using your Azure cheat sheet.
6. Week 6 — CI/CD & observability: GitHub Actions pipeline (test → build → push → deploy), Application Insights, and a security review against the Part 5 checklist.

What's included alongside this guide

- This document (FastAPI_Complete_Guide.docx) — reference material for every stage above.
- A runnable starter project (fastapi_production_starter.zip) implementing the structure from Section 5.1, with auth, database models, tests, Dockerfile and docker-compose already wired up — so you can start from a working skeleton instead of a blank file.

Next step: Ask me anytime to explain any single concept in more depth (e.g., "explain JWT refresh tokens" or "walk me through deploying the starter project to Azure Container Apps step by step") and I'll go deeper with a focused, beginner-friendly breakdown.

LEARNING FLOW: What/Why → Install → First App → Run → Path/Query → Request Body → Response/Status → Errors → Param Types → Depends → APIRouter → Middleware/CORS → Background Tasks → File Uploads → Async/Sync → Config

1 What & Why

- ▶ Modern, high-perf Python API framework on **type hints**.
- ▶ Built on **Starlette** (ASGI) + **Pydantic** (validation).
- ▶ **Fast**: async/await, near Node/Go speed.
- ▶ Type hints = validation + docs (no schema files).
- ▶ Auto docs: `/docs` (Swagger), `/redoc`.
- ▶ Built-in **Depends** DI; OpenAPI/JSON-Schema based.

2 Install

- ▶ Needs an ASGI server (**Uvicorn** dev).

```
# venv
python -m venv .venv
source .venv/bin/activate
# full install
pip install "fastapi[standard]"
# minimal
pip install fastapi uvicorn[standard]
```

3 First App

```
# main.py
from fastapi import FastAPI
app = FastAPI(title="My API",
              version="1.0.0")

@app.get("/")
def read_root():
    return {"message": "Hello!"}
```

4 Run

```
uvicorn main:app --reload
```

- ▶ `--reload` = auto-restart on save.
- ▶ App at `127.0.0.1:8000`.
- ▶ Swagger → `/docs`, ReDoc → `/redoc`.

5 Path & Query Params

- ▶ Source inferred from signature; bad type → auto HTTP 422.

```
from typing import Optional
@app.get("/items/{item_id}")
def read_item_id: int,
        q: Optional[str]=None,
        active: bool=True):
    # item_id=path, q/active=query
    return {"id": item_id}
```

6 Request Body

- ▶ A **Pydantic model** param = JSON body (POST).

```
from pydantic import BaseModel, Field
class Item(BaseModel):
    name: str
    price: float = Field(gt=0)
    tax: float | None = None

@app.post("/items/")
def create(item: Item):
    return item
```

7 Response & Status

- ▶ **response_model** filters output (hide secrets).
- ▶ Use separate IN / OUT models.

```
class UserIn(BaseModel):
    username: str; password: str
class UserOut(BaseModel):
    username: str

@app.post("/users/",
        response_model=UserOut,
        status_code=201)
def create(u: UserIn): return u
```

8 Error Handling

- ▶ **HTTPException** = built-in clean errors; no try/except needed.

```
from fastapi import HTTPException
@app.get("/items/{id}")
def get(id: str):
    if id not in db:
        raise HTTPException(
            404, detail="Not found")
    return db[id]
```

9 Parameter Types

Path	in {}: f(item_id:int)
Query	not in path: f(skip:int=0)
Body	Pydantic model: f(item:Item)
Header	Header(None) → reads HTTP header (snake → Hyphen)
Cookie	Cookie(None) → reads browser cookie

Best practice: one model per purpose (Create/Update/Response).

10 Dependency Injection

- ▶ **Depends()**: run a fn before endpoint, inject result.
- ▶ For DB sessions, auth, pagination; cached per request; chainable.

```
from fastapi import Depends
def pager(skip=0, limit=20):
    return {"skip":skip}
@app.get("/items/")
def ls(p: dict = Depends(pager)):
    return p
# enforce-only (no value):
# dependencies=[Depends(check)]
```

11 APIRouter

- ▶ Split endpoints per feature → maintainable.
- ▶ **prefix** adds path; **tags** group in docs.

```
# routers/items.py
from fastapi import APIRouter
router = APIRouter(
    prefix="/items", tags=["items"])
@router.get("/")
def ls(): return []
# main.py
app.include_router(router)
```

12 Middleware & CORS

- ▶ **Middleware**: runs on every req/resp (logging, timing).
- ▶ **CORS**: browser rule; **not** server security → pair with auth.

```
from fastapi.middleware.cors \
    import CORSMiddleware
app.add_middleware(CORSMiddleware,
    allow_origins=["https://x.com"],
    allow_methods=["*"])
@app.middleware("http")
async def m(request, call_next):
    r = await call_next(request)
    return r
```

- ▶ Before `call_next` = request; after = response.
- ▶ Stack order: last registered runs first; response reverses.

13 Background Tasks

- ▶ Return response now, run light job after (email/log).
- ▶ Heavy/durable work → Celery/RQ/Azure Queue.

```
from fastapi import BackgroundTasks
@app.post("/signup")
def signup(email: str,
        bg: BackgroundTasks):
    bg.add_task(send_mail, email)
    return {"ok": True}
```

14 File Uploads

- ▶ **UploadFile** = **File(...)** = mandatory file.

```
from fastapi import UploadFile, File
@app.post("/upload")
async def up(
    file: UploadFile = File(...)):
    data = await file.read()
    return {"name": file.filename,
            "size": len(data)}
```

15 Async vs Sync

Async DB (asyncpg)	async def + await
Sync DB (psycopg2)	def (threadpooled)
httpx.AsyncClient	async def + await
CPU-heavy / ML	def or worker/queue
▶ Sync endpoints auto-run in a threadpool.	

16 Config (§4.1)

- ▶ Never hard-code secrets. **BaseSettings** reads `env/.env`, validates, cached.

```
from functools import lru_cache
from pydantic_settings import \
    BaseSettings, SettingsConfigDict
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env")
    database_url: str
    secret_key: str
    algorithm: str = "HS256"
@lru_cache
def get_settings(): return Settings()
```